

CS2610: Computer Organization and Architecture

Lab

Lab 3: Stack and Recursion

16th February 2025

Objective

Write recursive programs in RISC-V assembly to understand argument passing, stack maintenance, and local variable allocation.

Question 1: Sum of Digits

Consider a non-negative integer N . You are required to compute the **sum of its digits** using **recursive** RISC-V assembly programs.

(a) Recursive Implementation (40 points)

Write a recursive RISC-V assembly program to compute the sum of digits of N .

- The value of N should be stored in memory.
- Recursion must be used. Use of loops is not allowed.
- Use the following recursive formulation:

$$\text{sumDigits}(N) = \begin{cases} N, & \text{if } N < 10 \\ (N \bmod 10) + \text{sumDigits}\left(\left\lfloor \frac{N}{10} \right\rfloor\right), & \text{otherwise} \end{cases}$$

- You may use the following RISC-V instructions for division and remainder:

```
div rd, rs1, rs2    # rd = rs1 / rs2    (integer division)
rem rd, rs1, rs2    # rd = rs1 % rs2    (remainder)
```

- The final result must be returned in register `a0`.
- Proper saving and restoring of registers is required.
- Maintain global variables to keep track of lowest and highest `sp` values. at the end, before returning to the main, ensure `a0 = sum of digits(N)` and `a1 = stack usage`.

(b) Bonus: Tail-Recursive Implementation

Rewrite the above program using **tail recursion**.

- Implement a tail-recursive function `sumDigitsTR`.
- The function must take two arguments:
 - * the number N
 - * an accumulator variable
- Use the following formulation:

$$\text{sumDigitsTR}(N, acc) = \begin{cases} acc + N, & \text{if } N < 10 \\ \text{sumDigitsTR}\left(\left\lfloor \frac{N}{10} \right\rfloor, acc + (N \bmod 10)\right), & \text{otherwise} \end{cases}$$

- No computation should be performed after the recursive call.
- Analyze the stack usage.
- Clearly indicate how arguments are passed and updated across recursive calls.
- Submit Screenshots of the output for $N = 1, 23, 345, 6789$.

Question 2: Printing Balanced Parentheses (60 Points)

Given a non-negative integer N , write a **recursive** RISC-V assembly program to print all valid combinations of balanced parentheses consisting of N pairs.

Problem Description

A string of parentheses is said to be **balanced** if:

- At no point does the number of closing parentheses exceed the number of opening parentheses.
- The total number of opening parentheses equals the total number of closing parentheses.

For example, when $N = 3$, the valid outputs are:

$((()))$, $((()))$, $(()())$, $(()())$, $(())()$

Hint:

The following C program describes the recursive logic required for this problem:

```
char str[100];

void printParens(int open, int close, int idx) {
    if (open == 0 && close == 0) {
        str[idx] = '\0';
        printf("%s\n", str);
        return;
    }
}
```

```

    if (open > 0) {
        str[idx] = '(';
        printParens(open - 1, close, idx + 1);
    }

    if (close > open) {
        str[idx] = ')';
        printParens(open, close - 1, idx + 1);
    }
}

int main() {
    int N = 3;
    printParens(N, N, 0);
    return 0;
}

```

Implementation Requirements

- The RISC-V assembly implementation must follow the above C logic (Understand it before implementing it).
- Recursion must be used. Use of loops is not allowed.
- The value of N must be stored in memory.
- The current parenthesis string can be maintained using memory. For example:

```
buffer: .space 21    # supports N <= 10 (2*N characters + null terminator)
```

- Proper saving and restoring of registers across recursive calls is required.
- Each valid parenthesis combination must be printed on a new line.
- You can use the C code to check your output.
- Submit Screenshots of the output of your assembly program for $N = 2, 3, 4$.
- To print a string you can make the write syscall in RISC-V as shown below - an example code file is also given:

```

# buffer contains the current parenthesis string
# idx contains the length of the string
# newline is the string - "\n"

# Null-terminate the string
la t0, buffer
add t0, t0, a2 # a2 holds idx
sb zero, 0(t0)

# Write the string to stdout
li a0, 1 # file descriptor (stdout)
la a1, buffer # address of string
mv a2, a2 # length of string (idx)
li a7, 64 # syscall number for write
ecall

```

```
# Print newline
li a0, 1
la a1, newline
li a2, 1
li a7, 64
ecall
```

Submission Instructions

1. The assignment must be completed individually.
2. The following artifacts must be submitted:
 - Screenshots of the outputs as specified for each question
 - Code files for each question
3. Compress all files into a single ZIP archive and submit it on Moodle.